TECH BLAZE CONSULTING

FEEDBACK ON NCCOE CONCEPT PAPER

Accelerating the Adoption of Software and AI Agent Identity and Authorization

From: Pratyush Sood, Principal Consultant — Tech Blaze Consulting Pty Ltd (Canberra, ACT, Australia)

To: AI-Identity@nist.gov

Reference: NCCoE Concept Paper, February 5, 2026

Prior engagement: NIST-2025-0035 (CAISI Request for Information)

1. About the Respondent

Tech Blaze Consulting Pty Ltd is an Australian cybersecurity consultancy based in Canberra, ACT. The principal consultant (Pratyush Sood) holds CISM, CISA, IRAP (Australian Cyber Security Centre), TOGAF Enterprise Architecture Practitioner, and Microsoft Azure Solutions Architect Expert certifications, with over 15 years of experience across IT infrastructure, security assessment, compliance, and enterprise architecture.

This feedback draws on direct experience conducting IRAP security assessments for Australian Government systems, designing enterprise architectures for federal agencies, and building IDProva — an open-source protocol (Apache 2.0) for verifiable AI agent identity that addresses many of the challenges outlined in the concept paper. IDProva is referenced throughout this response as a working example of one approach, not as the only approach.

2. General Questions and Use Cases

Q: What enterprise use-cases are organisations currently using agents for?

From our consulting practice across Australian Government and enterprise environments, we see four patterns emerging:

Workforce productivity agents — Agents integrated with calendars, email, and document management systems to draft communications, summarise meeting outcomes, and coordinate scheduling. These agents require delegated access to multiple data sources under a single user's authority.

Security operations agents — Agents that ingest SIEM alerts, correlate threat intelligence, and either recommend or take remediation actions. The identity challenge is acute here because these agents access highly sensitive security data and can take destructive actions (e.g., isolating hosts, blocking IPs).

CI/CD pipeline agents — Agents that deploy code, run tests, and manage infrastructure-as-code. These already exist in most organisations, but the shift to LLM-powered agents (e.g., coding assistants that commit directly) introduces non-deterministic decision-making that existing service account models weren't designed for.

Multi-agent orchestration — Architectures where a lead agent delegates subtasks to specialist worker agents. This is the use case that most urgently requires new identity and authorization primitives, because delegation chains create authority relationships that OAuth 2.0 alone doesn't model well.

Q: What risks worry you most about agents?

Three risks stand out from our assessment work:

Invisible privilege escalation through delegation — When Agent A delegates to Agent B, and Agent B delegates to Agent C, current systems have no mechanism to ensure that C's effective permissions are a subset of A's. Without structural scope narrowing, every delegation step is a potential privilege escalation.

Accountability gaps in multi-agent chains — When an agent takes an action, there is often no cryptographic trail linking that action back through the delegation chain to the human who authorised it. Application-layer logs are insufficient because they can be tampered with and they don't capture the authority relationship.

Shared credential anti-pattern — Most organisations currently give agents service account credentials or API keys with broad permissions. This is the equivalent of giving every contractor the master key — it violates least privilege and makes attribution impossible.

Q: What support are you seeing for new protocols such as MCP?

MCP adoption is accelerating rapidly. Anthropic's Model Context Protocol has become the de facto standard for connecting LLM-based agents to tools and data sources. However, the current MCP specification relies on OAuth 2.0/OIDC for authorization, which was designed for human-to-service authorization — not for agent-to-tool or agent-to-agent delegation chains. The gap is clear: MCP needs an authorization layer that understands agent-specific semantics like delegation depth, scope narrowing, and config attestation.

Q: How do AI agents differ from other forms of software agents?

The critical difference is non-determinism. Traditional software agents (cron jobs, microservices, CI/CD runners) execute predictable code paths — their behaviour is determined by their source code. AI agents use LLM reasoning to decide which tools to call, what parameters to use, and how to interpret results. This means their required permissions cannot be fully predicted at deployment time, which fundamentally challenges the "least

privilege" principle. We need authorization frameworks that can constrain the envelope of permitted actions without needing to enumerate every possible action in advance.

3. Identification

Q: How might agents be identified in an enterprise architecture? What metadata is essential for an AI agent's identity?

Agents need purpose-built identifiers that are structurally distinct from human user identities. Treating agents as "users with API keys" conflates two fundamentally different identity models. An agent's identity needs to carry metadata that has no analogue in human identity systems:

Model identifier — Which AI model (e.g., anthropic/claude-opus-4) powers the agent's reasoning

Runtime platform — The execution environment (e.g., a specific orchestration framework and version)

Configuration attestation — A cryptographic hash of the agent's system prompt, tool definitions, and runtime parameters, so verifiers can detect if the agent has been tampered with

Trust level — A graduated classification (from unverified through to continuously monitored) that determines what the agent is permitted to do

Capability declarations — What the agent can do (e.g., call MCP tools, delegate authority, communicate with other agents)

Maximum delegation depth — How many levels of sub-delegation the agent is permitted to create

W3C Decentralized Identifiers (DIDs) are a strong fit for agent identity because they are self-sovereign, cryptographically verifiable, and support extensible service metadata. In IDProva, we use the did:idprova: method with a format of did:idprova:<authority>:<agent-name>, where the authority is the namespace owner and the agent-name is locally unique. For domain-verified agents, the controller proves domain ownership via DNS TXT records.

Q: Should agent identity metadata be ephemeral or fixed?

Both. The core identity (DID, public key, organisational binding) should be persistent. But operational metadata — particularly configuration attestation and trust level — must be updatable. If an agent's system prompt changes (legitimately or through injection), its configuration hash changes, and downstream verifiers should be able to detect this drift and

make trust decisions accordingly. This is a critical mechanism for detecting certain classes of prompt injection attacks (see Section 7).

Q: Should agent identities be tied to specific hardware, software, or organisational boundaries?

We recommend a tiered approach. At higher trust levels, binding agent identity to hardware (TPM, HSM) provides stronger assurance. At lower trust levels, software-based key storage is pragmatic for rapid deployment. The following trust hierarchy has proven workable:

Trust Level	Identity Binding	Use Case
L0- Unverified	Self-asserted, no external verification	Development, testing
L1- Domain-Verified	DNS TXT record proves domain ownership	Production agents with known operator
L2- Org-Verified	Organisation identity confirmed by registry	Enterprise deployments
L3- Third-Party Attested	Independent assessor validates identity claims	High-assurance regulated environments
L4- Continuously Monitored	Real-time monitoring with automated trust demotion	Critical infrastructure, financial services

4. Authentication

Q: What constitutes strong authentication for an AI agent?

Strong agent authentication requires cryptographic proof tied to the agent's identity document. We recommend Ed25519 signatures (FIPS 186-5) as the classical baseline, with hybrid Ed25519 + ML-DSA-65 (FIPS 204) co-signatures for post-quantum readiness. An attacker must break both algorithms to forge a signature. IDProva implements this hybrid scheme from day one, with a migration roadmap through 2029 that progressively deprecates classical-only mode.

Q: How do we handle key management for agents? Issuance, update, and revocation?

Key lifecycle management for agents follows established PKI principles, but with shorter rotation cycles to account for the higher risk of autonomous systems:

Algorithm	Rotation Period	Mechanism
Ed25519	Every 90 days	DID Document update — add new key, transition period, remove old
ML-DSA-65	Every 180 days	Same mechanism — old and new keys coexist during transition
Emergency	Immediate	Upon suspected compromise — immediate key revocation

Key storage should follow a hierarchy based on the agent's trust level: HSM (FIPS 140-2 Level 2+) for high-assurance agents, TPM for device-bound agents, OS keychain for standard deployments, and AES-256-GCM encrypted files with Argon2id key derivation as a minimum baseline. All keys must be generated using cryptographically secure random number generators.

5. Authorization and Delegation

Q: How can zero-trust principles be applied to agent authorization?

Zero trust for agents means: never trust an agent's claim of authority — always verify cryptographically. In practice, this requires short-lived, scoped authorization tokens that are verified at every interaction point, not just at session establishment. IDProva's Delegation Authority Tokens (DATs) are JWS-encoded tokens (RFC 7515) with enforced time-to-live limits tied to trust level:

Trust Level	Maximum Token Lifetime
L0–L1 (Unverified / Domain-Verified)	24 hours
L2–L4 (Org-Verified through Continuously Monitored)	7 days

Q: How do we establish least privilege for an agent when its required actions are not fully predictable?

This is the central challenge for AI agent authorization. Unlike traditional software, AI agents use LLM reasoning to decide which actions to take, so you cannot enumerate every permitted action at deployment time. The solution is scope-based authorization using a structured grammar: scope = namespace ":" resource ":" action

This allows organisations to define the envelope of permitted actions (mcp:tool:filesystem:read) without needing to predict every specific file the agent will access. Wildcards enable controlled breadth (mcp:tool:filesystem:*) for all filesystem operations) while still enforcing boundaries.

Q: How do we handle delegation of authority for "on behalf of" scenarios?

Delegation is where existing identity standards fall shortest. OAuth 2.0 supports basic delegation (user grants access to an application), but it was not designed for multi-hop agent delegation chains where Agent A delegates to Agent B, which delegates to Agent C. The critical requirement is a scope narrowing rule: each child delegation must be a strict subset of its parent's scope. This prevents privilege escalation at every link in the chain.

Agent B cannot grant Agent C database write access because that exceeds Agent B's own authority. Additionally, constraints (rate limits, IP ranges, geographic restrictions, maximum action counts, config attestation requirements) must inherit and narrow through the chain — a child can never weaken a parent's constraint.

Q: What are the mechanisms for an agent to prove its authority to perform a specific action?

Authorization enforcement must happen outside the LLM reasoning boundary. If the LLM decides whether an action is authorized, then a prompt injection can bypass authorization. Instead, authorization should be enforced cryptographically at the protocol binding layer (MCP server, A2A gateway, HTTP endpoint) before any action executes. The LLM cannot bypass a failed cryptographic verification.

In IDProva's MCP binding, the server validates the DAT before executing any tool call. The verification chain is: decode JWS → verify signature(s) → resolve issuer's DID → verify delegation chain → extract effective scopes → check required scope against DAT. This is fully cryptographic and non-bypassable by the agent's LLM.

Revocation must also cascade through delegation chains. When a parent DAT is revoked, all child DATs must become invalid immediately. Verifiers should check revocation status for every DAT in the chain, not just the leaf token.

6. Auditing and non-repudiation

Q: How can we ensure that agents log their actions in a tamper-proof and verifiable manner?

Application-layer logs are insufficient for agent audit because they are mutable, they don't capture the authority relationship, and they can't prove who authorised an action. What's needed is a cryptographic audit trail with three properties: tamper evidence, delegation correlation, and non-repudiation.

IDProva's Action Receipts demonstrate one approach. Each agent action produces a signed receipt that forms a hash chain using BLAKE3. Each receipt captures who (agent DID), what (action type, target, parameters), when (ISO 8601 UTC with millisecond precision), where (environment, session), and outcome (success, failure, error). The receipt is signed by the agent's private key, and the delegation field references the DAT that authorised the action — walking the chain traces authority from any action back to the human principal.

Tampering with any receipt breaks the hash chain. Inserting or deleting receipts is detected by sequence number gaps. The combination of hash chaining + per-receipt signatures + delegation correlation provides complete non-repudiation.

Q: How do we ensure non-repudiation for agent actions and binding back to human authorization?

The critical requirement is delegation correlation. Every audit record must reference the authorization token (e.g., DAT jti) that permitted the action. Walking the delegation chain from that token traces authority through every intermediary back to the original human principal. This is how an assessor answers: "Who authorised this agent to take this action at this time?"

This approach maps directly to existing compliance frameworks:

NIST 800-53	Description	How Agent Receipts Satisfy
AU-2	Auditable Events	Every agent action produces a receipt
AU-3	Content of Audit Records	Who, what, when, where, outcome in every receipt
AU-8	Time Stamps	ISO 8601 UTC with milliseconds in signed payload
AU-9	Protection of Audit Information	Hash chain + Ed25519 signatures
AU-10	Non-repudiation	Per-receipt signature + delegation chain to human principal
AU-12	Audit Record Generation	Protocol-native, enforced via DAT constraints
IA-2	Identification & Authentication	DID-based identity in every receipt
AC-6	Least Privilege	Scoped delegation + constraints

Equivalent mappings exist for the Australian ISM (ISM-0585, ISM-0988, ISM-0580, ISM-1405) and SOC 2 Trust Services Criteria (CC6.1, CC6.2, CC6.3, CC7.2).

7. Prompt Injection Prevention and Mitigation

Q: What controls help prevent both direct and indirect prompt injections?

Prompt injection is a multi-layer problem that requires defenses at multiple levels. From an identity and authorization perspective, two controls are particularly relevant:

Configuration attestation — An agent's system prompt, tool definitions, model identifier, and runtime parameters should be hashed at deployment time (we use BLAKE3) and published in the agent's identity document. If a prompt injection modifies the system prompt or tool definitions, the hash changes, and downstream verifiers can detect the drift. DATs can include a requiredConfigAttestation constraint that rejects requests from agents whose config no longer matches the attested baseline.

Authorization outside the LLM — If the LLM decides whether an action is authorized, then a successful prompt injection can bypass authorization entirely. Moving authorization enforcement to the protocol binding layer (MCP server, HTTP gateway) ensures that even a fully compromised LLM cannot execute actions outside the agent's cryptographically defined scope.

Q: After prompt injection occurs, what controls/practices can minimise the impact?

Three mechanisms limit blast radius:

Scope constraints on DATs — Even if an agent's reasoning is compromised, it cannot take actions outside the scope defined in its delegation token. Rate limits, maximum action counts, and IP restrictions further bound the damage.

Trust level demotion — Agents operating at higher trust levels (e.g., L4 Continuously Monitored) can be automatically demoted to a lower trust level if monitoring detects policy violations, including config drift that may indicate injection.

Cascading revocation — Revoking a compromised agent's DAT immediately invalidates all child DATs in the delegation chain, cutting off the entire sub-tree of agents operating under the compromised agent's authority.

> **A note on honesty:** The identity and attestation layer provides **detection** of prompt injection (via config drift), not **prevention** of adversarial prompts. Runtime input filtering, output sanitization, and LLM-layer defenses remain necessary complements. What the identity layer establishes is *whether the agent is still the agent it claims to be* — a necessary but not sufficient component of a complete prompt injection defense.

8. Standards and Interoperability Considerations

Q: What standards exist, or are emerging, to support identity and access management of agents?

The concept paper correctly identifies OAuth 2.0/OIDC, SPIFFE/SPIRE, SCIM, and NGAC as relevant standards. From our work, we'd add several observations:

OAuth 2.0/OIDC is necessary but not sufficient — OAuth handles human-to-service authorization well. But it was not designed for multi-hop delegation chains, scope narrowing, config attestation, or hash-chained audit trails. An OAuth token can bootstrap an agent-specific token (e.g., a human authenticates via OIDC, then issues a scoped delegation token to their agent), but OAuth alone does not provide the agent-specific semantics the concept paper describes.

SPIFFE/SPIRE provides workload identity, not agent identity — SPIFFE SVIDs identify workloads in infrastructure. They don't model delegation chains, scoped permissions, or audit trails. SPIFFE and agent identity should coexist: SPIFFE identifies the workload, the agent identity layer identifies what authority the agent has and who granted it.

W3C DIDs are underexplored for agent identity — The concept paper doesn't mention DIDs, but they are a natural fit. DIDs are self-sovereign, cryptographically verifiable, support extensible service metadata, and are already a W3C Recommendation (2022). They provide a standards-based foundation for agent identity without requiring a new identifier standard.

MCP needs an agent-aware authorization layer — The current MCP specification defers authorization to OAuth/OIDC. This leaves a gap for agent-specific authorization semantics. Protocol bindings that validate agent delegation tokens at the MCP server level (before tool execution) would close this gap.

The standards IDProva builds on:

Standard	Usage
W3C DIDs	Agent identity documents
JWS (RFC 7515)	Delegation token encoding and signature
FIPS 186-5 / FIPS 204	Ed25519 (classical) + ML-DSA-65 (post-quantum) signatures
BLAKE3	Receipt hash chains, configuration attestation

RFC 8785 (JCS)	Canonical serialization for deterministic hashing

Post-Quantum Readiness

We strongly recommend that the NCCoE demonstration project require post-quantum readiness in any agent identity solution evaluated. Harvest-now-decrypt-later attacks are a real threat to long-lived audit trails. IDProva uses a hybrid Ed25519 + ML-DSA-65 scheme from day one, with a migration roadmap that progressively deprecates classical-only mode through 2029.

9. Technology Collaboration Interest

Tech Blaze Consulting would welcome the opportunity to contribute to the NCCoE demonstration project. IDProva is licensed under Apache 2.0 and is available for independent evaluation without licensing agreements.

Available Technology

Component	Description	Status
Rust SDK	Reference implementation (5 crates: core, verify, registry, middleware, cli)	205 tests passing
Python SDK	PyO3 bindings (pip install idprova)	Built from Rust core
TypeScript SDK	napi-rs bindings (@idprova/core)	Built from Rust core
CLI Tool	9 commands covering full agent lifecycle	Ready for integration
Self-Hosted Registry	Docker image for AID registration and DAT management	Production-ready

Community of Interest Participation

Tech Blaze Consulting requests membership in the NCCoE Community of Interest for this project. As an international technology contributor with cross-jurisdictional compliance experience (NIST 800-53, Australian ISM, SOC 2), we can provide protocol design perspective, multi-framework compliance mapping, and practical assessment experience from IRAP engagements.

10. STRIDE Threat Analysis for Agentic Architectures

The NCCoE concept paper focuses on identity and authorization controls, but a complete security analysis requires understanding **where trust boundaries lie** in agentic architectures and what threats emerge at each boundary. The STRIDE framework (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) provides a structured lens, though recent research [1][2] demonstrates that traditional STRIDE analysis alone is insufficient for agentic systems due to their non-deterministic, multi-step, and stateful nature.

Trust Boundaries in Agentic Architectures

In a typical agentic architecture as depicted in the concept paper (Figure 1), we identify four critical trust boundaries:

TB1: Human → Agent — The boundary where a human principal delegates authority to an AI agent. This is where the delegation chain begins and where the initial scope of permitted actions must be defined.

TB2: Agent → LLM Reasoning — The boundary between the agent runtime and the LLM that provides reasoning. This is where prompt injection crosses from untrusted input into the agent's decision-making process.

TB3: Agent → Tools/Resources — The boundary where an agent accesses external tools (MCP servers, APIs, databases). Authorization must be enforced here, outside the LLM reasoning boundary.

TB4: Agent → Agent — The boundary where agents delegate to or communicate with other agents. This is where delegation chains extend and where scope narrowing must be enforced structurally.

STRIDE Mapping to Agentic Trust Boundaries

The following table maps each STRIDE threat category to the specific risks at agentic trust boundaries and demonstrates how protocol-level controls (using IDProva as a reference implementation) address each threat:

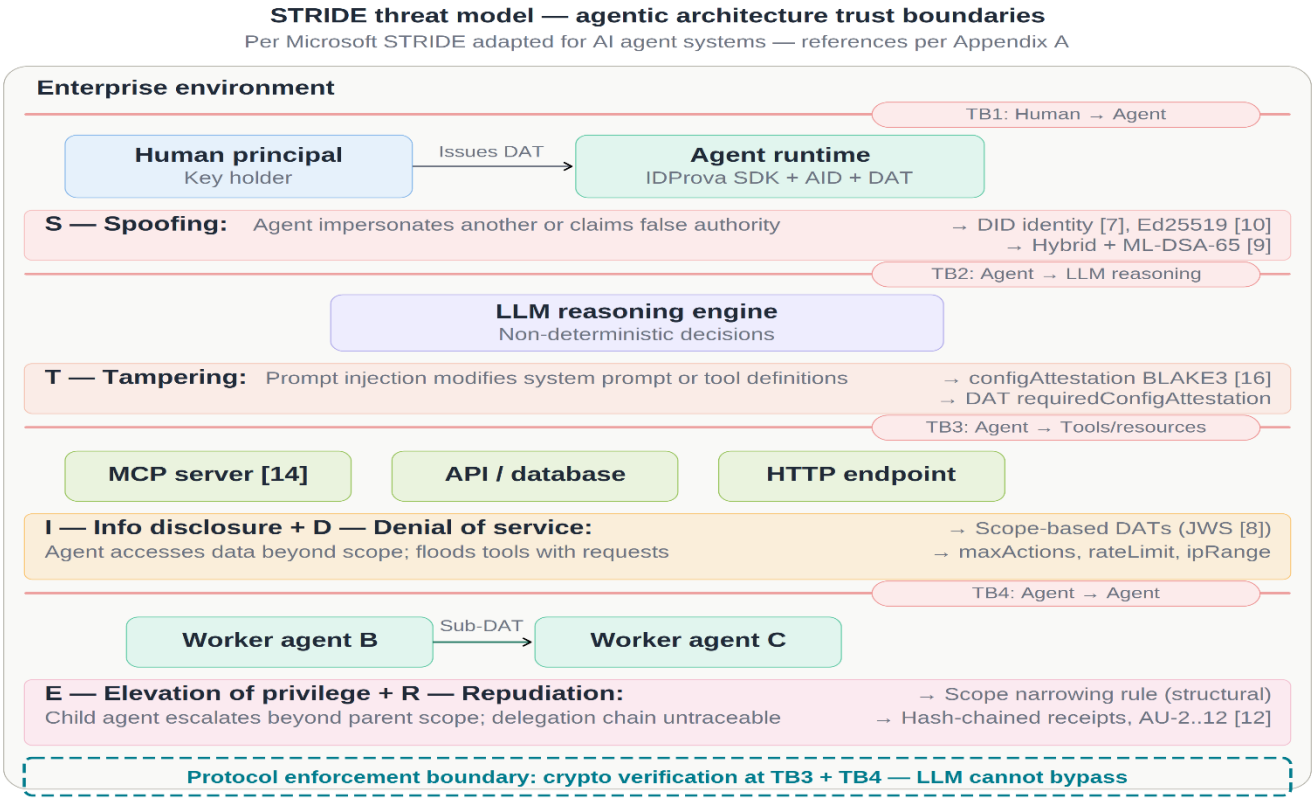

Figure 1: STRIDE threat model for agentic architectures showing trust boundaries (TB1–TB4) and protocol-level mitigations. Adapted from Hernan et al. (2006) with STRIDE+LM extensions [18] and MAESTRO [3] for AI agent systems. Reference numbers correspond to Appendix A.

STRIDE Category	Agentic Threat	Trust Boundary	IDProva Mitigation
Spoofing	An agent impersonates another agent or claims false authority in a delegation chain	TB1, TB4	DID-based cryptographic identity (did:idprova:) with Ed25519 signature verification. Each agent's identity is bound to a key pair published in its AID Document.
Tampering	Prompt injection modifies an agent's system prompt or tool definitions, altering its behaviour	TB2	configAttestation field in AID Documents stores a BLAKE3 hash of agent config at deployment. DATs can require matching attestation, rejecting tampered agents.
Repudiation	An agent denies having taken an action, or the delegation chain cannot be traced back to the authorising human	TB1, TB3, TB4	Hash-chained Action Receipts signed by the agent's private key. Delegation correlation links every action to the DAT jti and ultimately to the human principal.
Information Disclosure	An agent accesses data beyond its authorised scope, or receipts leak sensitive parameters	TB3	Scope-based DATs restrict access to specific namespaces, resources, and actions. Constraints (IP ranges, geo-restrictions) further limit data exposure.
Denial of Service	A compromised agent floods tools with requests, or creates an unbounded delegation chain that exhausts resources	TB3, TB4	DAT constraints include maxActions (per-token action limit), rateLimit (actions per time window), and maxRedelegationDepth (bounds chain depth).
Elevation of Privilege	Agent B receives broader permissions than Agent A intended when delegating, or a child agent escalates beyond its parent's scope	TB4	Structural scope narrowing rule: each child DAT must be a strict subset of its parent's scope. Constraints inherit and narrow — a child can never weaken a parent's constraint.

As recent literature [1][3] observes, STRIDE alone does not fully capture agentic threats such as goal manipulation across reasoning cycles or persistent memory corruption. The Cloud Security Alliance's MAESTRO framework [2] extends traditional threat models with agentic-specific layers. However, the identity and authorization controls described in this feedback

— cryptographic identity, scoped delegation with structural narrowing, and hash-chained audit trails — address the foundational trust boundary threats that underpin both the traditional STRIDE categories and the emerging agentic-specific threat models.

Where the Protocol Boundary Lies

A practical protocol for agent identity and authorization must enforce controls at the protocol binding layer (TB3 and TB4), not inside the LLM reasoning boundary (TB2). This is the fundamental architectural decision that distinguishes cryptographic authorization from prompt-based authorization:

Protocol-layer enforcement (TB3/TB4) — The MCP server, A2A gateway, or HTTP endpoint validates the agent's DAT before any action executes. The LLM cannot bypass this because the verification is cryptographic (JWS signature verification), not reasoning-based.

LLM-layer enforcement (TB2) — Relying on the LLM to decide whether an action is authorized is inherently vulnerable to prompt injection. A successful injection can override the LLM's reasoning about permissions.

IDProva demonstrates that the protocol boundary should sit at TB3 and TB4 — where the agent meets external systems and other agents. The LLM at TB2 handles reasoning; the protocol layer handles authorization. This separation of concerns is what makes the system non-bypassable by design.

TECH BLAZE CONSULTING

Appendix A: References

[1] Booth, H., Fisher, B., Galluzzo, R. & Roberts, J. (2026). "Accelerating the Adoption of Software and AI Agent Identity and Authorization." NIST National Cybersecurity Center of Excellence, Concept Paper (Draft), February 2026. Available: https://www.nccoe.nist.gov/sites/default/files/2026-02/accelerating-the-adoption-of-software-and-ai-agent-identity-and-authorization-concept-paper.pdf

[2] South, T. et al. (2025). "Identity Management for Agentic AI: The New Frontier of Authorization, Authentication, and Security for an AI Agent World." OpenID Foundation Whitepaper, October 2025. Prepared in collaboration with Stanford's Loyal Agents Initiative and WorkOS. Available: https://openid.net/wp-content/uploads/2025/10/Identity-Management-for-Agentic-AI.pdf

[3] Cloud Security Alliance (2025). "Agentic AI Threat Modeling Framework: MAESTRO." CSA Blog, February 2025. Available: https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

[4] Jamil, S. et al. (2025). "Securing Agentic AI: A Comprehensive Threat Model and Mitigation Framework for Generative AI Agents." arXiv:2504.19956, May 2025. Available: https://arxiv.org/abs/2504.19956

[5] OWASP (2025). "Agentic AI Threats and Mitigations." OWASP Top 10 for LLM Applications & Generative AI, Version 1.0, February 2025. Available: https://genai.owasp.org/

[6] Leo, M., Tan, F., Miao, T. et al. (2026). "From Threat to Trust: Assessing Security Risks of Agentic AI Systems." International Journal of Information Security, 25, 23. DOI: https://doi.org/10.1007/s10207-025-01185-y

[7] W3C (2022). "Decentralized Identifiers (DIDs) v1.0." W3C Recommendation, 19 July 2022. Available: https://www.w3.org/TR/did-core/

[8] Jones, M. & Sakimura, N. (2015). "JSON Web Signature (JWS)." IETF RFC 7515, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7515

[9] NIST (2024). "Module-Lattice-Based Digital Signature Standard." FIPS 204 (ML-DSA), August 2024. Available: https://csrc.nist.gov/pubs/fips/204/final

[10] Josefsson, S. & Liusvaara, I. (2017). "Edwards-Curve Digital Signature Algorithm (EdDSA)." IETF RFC 8032, January 2017. Available: https://www.rfc-editor.org/rfc/rfc8032

[11] NIST (2020). "Zero Trust Architecture." NIST SP 800-207, August 2020. Available: https://csrc.nist.gov/pubs/sp/800/207/final

[12] Joint Task Force (2020). "Security and Privacy Controls for Information Systems and Organizations." NIST SP 800-53 Rev. 5, September 2020 (updated December 2020). Available: https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

[13] Australian Cyber Security Centre (2025). "Australian Government Information Security Manual." ISM, September 2025. Available: https://www.cyber.gov.au/resources-business-and-government/essential-cyber-security/ism

[14] Anthropic (2025). "Model Context Protocol Specification." Available: https://modelcontextprotocol.io/

[15] NIST (2025). "AI Agent Standards Initiative — Request for Information." Center for AI Standards and Innovation (CAISI), January 2026. Available: https://www.nist.gov/caisi/ai-agent-standards-initiative

[16] O'Connor, J. et al. (2023). "BLAKE3: One Function, Fast Everywhere." Available: https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[17] Schneider, C. (2026). "Threat Modeling Agentic AI: A Scenario-Driven Approach." Blog, February 2026. Available: https://christian-schneider.net/blog/threat-modeling-agentic-ai/

[18] Cloud Security Alliance (2025). "Agentic AI Security: New Dynamics, Trusted Foundations." CSA Blog, December 2025. STRIDE+LM extension for AI agent threats. Available: https://cloudsecurityalliance.org/blog/2025/12/18/agentic-ai-security-new-dynamics-trusted-foundations

Pratyush Sood

Principal Consultant — CISM, CISA, IRAP, TOGAF, Azure Solutions Architect Expert

Tech Blaze Consulting Pty Ltd

techblaze.com.au • idprova.dev • info@techblaze.com.au • +61 435 009 423